Algorithmics and Data Structures

Session 6
Dynamic Programming

Ali Ismail-Fawaz

ENSISA, Université Haute-Alsace

Dynamic Programming: Context & Impact

Where it came from:

- Introduced by **Richard Bellman**, an **American mathematician**, in the 1950s
- Developed at the RAND Corporation (USA) for operations research and optimization
- The name *"dynamic programming"* was chosen to make it sound practical and appealing to funding agencies

Why it matters:

- Converts exponential-time recursion into efficient polynomial-time solutions
- Core technique in algorithms, machine learning, bioinformatics, economics, and AI

Real-life example:

- *DNA/Protein sequence alignment* (edit distance / Smith–Waterman)
- Enables modern genomics: comparing genomes in hours instead of centuries
- Critical in personalized medicine, evolutionary biology, and pandemic response

"Those who cannot remember the past are condemned to repeat it."

— George Santayana, The Life of Reason (1905)

Note for class

This quotation is often associated by analogy with Dynamic Programming, since DP "remembers" past computations to avoid repeating work. While Bellman himself may not have cited Santayana, the connection is widely used in teaching.

Contents

Core Principles of Dynamic Programming

Longest Increasing Subsequence (LIS)

Shortest Path

Edit Distance (Levenshtein)

When to Use Dynamic Programming?

The Main Idea

Dynamic Programming (DP) breaks a problem into **smaller subproblems** and **reuses their solutions** instead of solving them repeatedly.

1. Overlapping Subproblems

The same subproblems appear **multiple times**.

Example: Fibonacci

$$F(n) = F(n-1) + F(n-2)$$

Computing $F(n-2)$ is needed in several recursive branches.

2. Optimal Substructure

An **optimal solution** can be constructed from optimal solutions to smaller subproblems.

Example: Shortest Path
A subpath of a shortest path is itself a shortest path.

DP vs. Greedy: What Is the Difference?

- **Greedy:** Makes the locally best choice at each step, without exploring alternatives.
- **DP:** Evaluates different choices and **reuses subproblem results** to find the global optimum.

Dynamic Programming: The Fibonacci Problem

Problem

Given $n \geq 0$, compute the n-th Fibonacci number:

$$F(n) = \begin{cases} n & \text{if } n \leq 1, \\ F(n-1) + F(n-2) & \text{otherwise.} \end{cases}$$

Example

$$0,\ 1,\ 1,\ 2,\ 3,\ 5,\ 8, \ldots$$

For $n = 5$:

$$F(5) = F(4) + F(3) = 5$$

Naive Recursive Algorithm

```
Fibonacci(n)
  If n ≤ 1: Return n
  Else: Return Fibonacci(n-1) + Fibonacci(n-2)
```

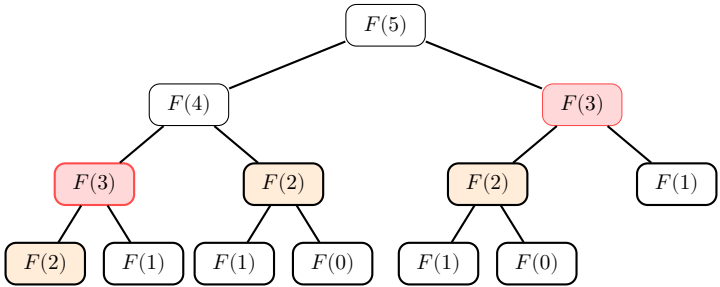
The Problem with Naive Recursion

The same subproblems are computed **multiple times**, leading to **exponential time complexity** $\mathcal{O}(2^n)$.

Why Is Naive Recursion Inefficient?

Recursive Fibonacci

Each call to $F(n)$ generates two recursive calls: $F(n-1)$ and $F(n-2)$, until reaching a base case.



The Problem: Repeated Computations

- $F(3)$ and $F(2)$ are computed **multiple times**.
- The number of recursive calls grows **exponentially**: $\mathcal{O}(2^n)$ time.
- **DP solution:** Compute each distinct subproblem once and reuse its result.

Two Ways to Apply Dynamic Programming

The Main Idea

Both methods solve each distinct subproblem **only once** and **reuse its result** instead of recomputing it.

1. Top-Down (Memoization)

- Start from the **original problem**.
- Solve smaller subproblems **recursively**.
- **Store** computed results in a cache.
- Reuse results when needed again.

Example: $F(5)$

$$F(5) \to F(4) \to F(3)$$

Recursive calls + cache

2. Bottom-Up (Tabulation)

- Start from the **smallest subproblems**.
- Solve larger subproblems **iteratively**.
- **Store** results in a table.
- Build up to the final answer.

Example: $F(5)$

$$F(0) \to F(1) \to \cdots \to F(5)$$

Iteration + table

Key Difference

Top-Down: Solve subproblems *on demand* using recursion.

Bottom-Up: Solve subproblems in a *predefined order*.

Solving Fibonacci with Top-Down DP

Top-Down DP Fibonacci(n, DP)

Input: n, array DP of size $n+1$ filled with -1
Output: n^{th} Fibonacci number
Instructions:

1. If $n \leq 1$
   Return n
2. If $DP[n] \neq -1$
   Return $DP[n]$
3. $DP[n] \leftarrow Fibonacci(n-1, DP) + Fibonacci(n-2, DP)$
4. Return $DP[n]$

Complexity

Time: $\mathcal{O}(n)$ **Space:** $\mathcal{O}(n)$ (cache + recursion stack)

Solving Fibonacci with Bottom-Up DP

Bottom-Up DP Fibonacci(n)

Input: n
Output: n^{th} Fibonacci number
Instructions:

1. $DP \leftarrow$ Array of size $n+1$ filled with 0
2. If $n \leq 1$
   Return n
3. $DP[1] \leftarrow 1$
4. For i from 2 to n do
   $DP[i] \leftarrow DP[i-1] + DP[i-2]$
5. Return $DP[n]$

Complexity

Time: $\mathcal{O}(n)$ **Space:** $\mathcal{O}(n)$ (DP table, no recursion stack)

Even More Optimization: Constant Space

Space-Optimized Fibonacci(n)

Input: n
Output: n^{th} Fibonacci number
Instructions:

1. If $n \leq 1$
   Return n
2. $prev2 \leftarrow 0$, $prev1 \leftarrow 1$
3. For i from 2 to n do
   a. $curr \leftarrow prev1 + prev2$
   b. $prev2 \leftarrow prev1$
   c. $prev1 \leftarrow curr$
4. Return $prev1$

Complexity

Time: $\mathcal{O}(n)$ **Space:** $\mathcal{O}(1)$ Same time complexity, but **constant auxiliary space**.

Dynamic Programming: Key Takeaways

The Main Idea

Dynamic Programming solves problems by breaking them into **smaller subproblems**, storing their solutions, and **avoiding repeated computations**.

When to Use DP?

- **Overlapping subproblems**
- **Optimal substructure**
- Solutions can be expressed using a **recurrence**

How to Apply DP?

- **Top-Down:** Recursion + Memoization
- **Bottom-Up:** Iteration + Tabulation
- Optimize memory when possible

A Key Problem-Solving Strategy

Many problems can be **transformed into well-known DP problems**.
Recognizing these patterns allows us to **reuse known algorithms** instead of designing solutions from scratch.

Contents

Core Principles of Dynamic Programming

Longest Increasing Subsequence (LIS)

Shortest Path

Edit Distance (Levenshtein)

Longest Increasing Subsequence (LIS)

Problem

Given an array A of n integers, find the **length** of the longest subsequence whose elements are **strictly increasing**.
A subsequence preserves the original order, but its elements do **not need to be contiguous**.

Example 1

Input:

$$A = [3, 10, 2, 1, 20]$$

LIS:

$$[3, 10, 20]$$

Output: 3

Example 2

Input:

$$A = [10, 9, 2, 5, 3, 7]$$

LIS:

$$[2, 3, 7]$$

Output: 3

Important: Subsequence $\neq$ Subarray

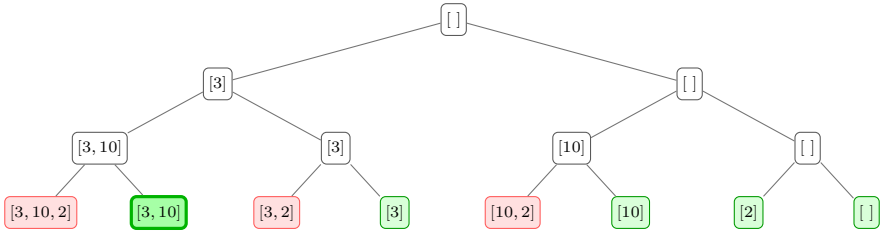
A **subarray** requires consecutive elements, whereas a **subsequence** can skip elements while preserving their original order.

LIS: Naive Solution (Brute Force)

Approach

For each element, choose to **include** or **exclude** it. Explore all subsequences and keep the longest increasing one.

Example: $A = [3, 10, 2]$



Complexity

Time: $\mathcal{O}(n2^n)$ **Space:** $\mathcal{O}(n)$
Result: LIS $= [3, 10]$, length $= 2$.

LIS: Dynamic Programming Solution

Bottom-Up DP

Let $L[i]$ be the length of the **longest increasing subsequence ending at index** i.

$$L[i] = 1 + \max_{\substack{1 \le j < i \\ A[j] < A[i]}} L[j] \qquad \text{(or 1 if no such } j\text{)}$$

Example: $A = [3, 10, 2, 11]$

i	1	2	3	4
$A[i]$	3	10	2	11
$L[i]$	1	1	1	1

Step-by-Step Computation

Initialize $L = [1, 1, 1, 1]$.

Result & Complexity

LIS: $\max(L) = 3$
Time: $\mathcal{O}(n^2)$
Space: $\mathcal{O}(n)$

LIS: Dynamic Programming Solution

Bottom-Up DP

Let $L[i]$ be the length of the **longest increasing subsequence ending at index** i.

$$L[i] = 1 + \max_{\substack{1 \le j < i \\ A[j] < A[i]}} L[j] \qquad \text{(or 1 if no such } j\text{)}$$

Example: $A = [3, 10, 2, 11]$

i	1	2	3	4
$A[i]$	3	10	2	11
$L[i]$	1	2	1	1

Step-by-Step Computation

$i = 2,\ j = 1$: $3 < 10$ (true)
$L[2] \leftarrow \max(1, 1 + 1) = 2$

Result & Complexity

LIS: $\max(L) = 3$
Time: $\mathcal{O}(n^2)$
Space: $\mathcal{O}(n)$

LIS: Dynamic Programming Solution

Bottom-Up DP

Let $L[i]$ be the length of the **longest increasing subsequence ending at index** i.

$$L[i] = 1 + \max_{\substack{1 \le j < i \\ A[j] < A[i]}} L[j] \qquad \text{(or 1 if no such } j\text{)}$$

Example: $A = [3, 10, 2, 11]$

i	1	2	3	4
$A[i]$	3	10	2	11
$L[i]$	1	2	1	1

Step-by-Step Computation

$i = 3,\ j = 1$: $3 < 2$ (false)
No update: $L[3] = 1$

Result & Complexity

LIS: $\max(L) = 3$
Time: $\mathcal{O}(n^2)$
Space: $\mathcal{O}(n)$

LIS: Dynamic Programming Solution

Bottom-Up DP

Let $L[i]$ be the length of the **longest increasing subsequence ending at index** i.

$$L[i] = 1 + \max_{\substack{1 \le j < i \\ A[j] < A[i]}} L[j] \qquad \text{(or 1 if no such } j\text{)}$$

Example: $A = [3, 10, 2, 11]$

i	1	2	3	4
$A[i]$	3	10	2	11
$L[i]$	1	2	1	1

Step-by-Step Computation

$i = 3,\ j = 2$: $10 < 2$ (false)
No update: $L[3] = 1$

Result & Complexity

LIS: $\max(L) = 3$
Time: $\mathcal{O}(n^2)$
Space: $\mathcal{O}(n)$

LIS: Dynamic Programming Solution

Bottom-Up DP

Let $L[i]$ be the length of the **longest increasing subsequence ending at index** i.

$$L[i] = 1 + \max_{\substack{1 \le j < i \\ A[j] < A[i]}} L[j] \qquad \text{(or 1 if no such } j\text{)}$$

Example: $A = [3, 10, 2, 11]$

i	1	2	3	4
$A[i]$	3	10	2	11
$L[i]$	1	2	1	2

Step-by-Step Computation

$i = 4,\ j = 1\text{:}\ 3 < 11$ (true)
$L[4] \leftarrow \max(1, 1 + 1) = 2$

Result & Complexity

LIS: $\max(L) = 3$
Time: $\mathcal{O}(n^2)$
Space: $\mathcal{O}(n)$

LIS: Dynamic Programming Solution

Bottom-Up DP

Let $L[i]$ be the length of the **longest increasing subsequence ending at index** i.

$$L[i] = 1 + \max_{\substack{1 \le j < i \\ A[j] < A[i]}} L[j] \qquad \text{(or 1 if no such } j\text{)}$$

Example: $A = [3, 10, 2, 11]$

i	1	2	3	4
$A[i]$	3	10	2	11
$L[i]$	1	2	1	3

Step-by-Step Computation

$i = 4,\ j = 2$: $10 < 11$ (true)
$L[4] \leftarrow \max(2, 1 + 2) = 3$

Result & Complexity

LIS: $\max(L) = 3$
Time: $\mathcal{O}(n^2)$
Space: $\mathcal{O}(n)$

LIS: Dynamic Programming Solution

Bottom-Up DP

Let $L[i]$ be the length of the **longest increasing subsequence ending at index** i.

$$L[i] = 1 + \max_{\substack{1 \le j < i \\ A[j] < A[i]}} L[j] \qquad \text{(or 1 if no such } j\text{)}$$

Example: $A = [3, 10, 2, 11]$

i	1	2	3	4
$A[i]$	3	10	2	11
$L[i]$	1	2	1	3

Step-by-Step Computation

$i = 4,\ j = 3$: $2 < 11$ (true)
$L[4] \leftarrow \max(3, 1 + 1) = 3$

Result & Complexity

LIS: $\max(L) = 3$
Time: $\mathcal{O}(n^2)$
Space: $\mathcal{O}(n)$

LIS: Bottom-Up DP Algorithm

Bottom-Up LIS

Input: Array A of size n $(n \geq 1)$
Output: Length of the longest increasing subsequence
Instructions:

1. $L \leftarrow [1] * n$
2. For i from 2 to n do
3. For j from 1 to $i-1$ do
4. If $A[j] < A[i]$ then
5. $L[i] \leftarrow \max(L[i], 1 + L[j])$
6. Return $\max(L[1], \ldots, L[n])$

Time Complexity

$$\sum_{i=2}^{n}(i-1) = \frac{n(n-1)}{2}$$

$$\boxed{\mathcal{O}(n^2)}$$

Space Complexity

Store the DP array L containing n values.

$$\boxed{\mathcal{O}(n)}$$

LIS: Recovering the Actual Subsequence in $\mathcal{O}(n \log n)$

Key Idea: Remember the Predecessors

Maintain $T[k]$: the index of the **smallest ending value** for a subsequence of length k, and $P[i]$: the index of the **predecessor** of $A[i]$. Binary search is performed on $A[T[k]]$.

Example: $A = [3, 10, 2, 11]$

A	3	10	2	11
T	1			
P	$-$			

Updating T and P

Process $A[1] = 3$

Start a subsequence of length 1.

$$T = [1], \qquad P[1] = -$$

Result & Complexity

LIS: $[3, 10, 11]$ **Length:** 3 **Time:** $\mathcal{O}(n \log n)$ **Space:** $\mathcal{O}(n)$

LIS: Recovering the Actual Subsequence in $\mathcal{O}(n \log n)$

Key Idea: Remember the Predecessors

Maintain $T[k]$: the index of the **smallest ending value** for a subsequence of length k, and $P[i]$: the index of the **predecessor** of $A[i]$. Binary search is performed on $A[T[k]]$.

Example: $A = [3, 10, 2, 11]$

A	3	10	2	11
T	1	2		
P	–	1		

Updating T and P

Process $A[2] = 10$
Since $3 < 10$, extend the subsequence.

$$T = [1, 2], \qquad P[2] = 1$$

Result & Complexity

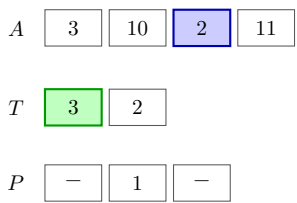
LIS: $[3, 10, 11]$ **Length:** 3 **Time:** $\mathcal{O}(n \log n)$ **Space:** $\mathcal{O}(n)$

LIS: Recovering the Actual Subsequence in $\mathcal{O}(n \log n)$

Key Idea: Remember the Predecessors

Maintain $T[k]$: the index of the **smallest ending value** for a subsequence of length k, and $P[i]$: the index of the **predecessor** of $A[i]$. Binary search is performed on $A[T[k]]$.

Example: $A = [3, 10, 2, 11]$



Updating T and P

Process $A[3] = 2$

Replace the best ending value for length 1.

$$T = [3, 2], \qquad P[3] = -$$

The predecessor $P[2] = 1$ is preserved.

Result & Complexity

LIS: $[3, 10, 11]$ **Length:** 3 **Time:** $\mathcal{O}(n \log n)$ **Space:** $\mathcal{O}(n)$

LIS: Recovering the Actual Subsequence in $\mathcal{O}(n \log n)$

Key Idea: Remember the Predecessors

Maintain $T[k]$: the index of the **smallest ending value** for a subsequence of length k, and $P[i]$: the index of the **predecessor** of $A[i]$. Binary search is performed on $A[T[k]]$.

Example: $A = [3, 10, 2, 11]$

A	3	10	2	11
T	3	2	4	
P	–	1	–	2

Updating T and P

Process $A[4] = 11$

Extend a subsequence of length 2.

$$T = [3, 2, 4], \qquad P[4] = 2$$

Result & Complexity

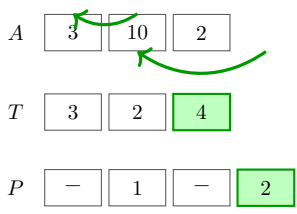
LIS: $[3, 10, 11]$ **Length:** 3 **Time:** $\mathcal{O}(n \log n)$ **Space:** $\mathcal{O}(n)$

LIS: Recovering the Actual Subsequence in $\mathcal{O}(n \log n)$

Key Idea: Remember the Predecessors

Maintain $T[k]$: the index of the **smallest ending value** for a subsequence of length k, and $P[i]$: the index of the **predecessor** of $A[i]$. Binary search is performed on $A[T[k]]$.

Example: $A = [3, 10, 2, 11]$



Updating T and P

Reconstruct the LIS

Start from $T[3] = 4$ and follow predecessors:

$$4 \to 2 \to 1$$

Reverse the values:

$$\boxed{[3, 10, 11]}$$

Result & Complexity

LIS: $[3, 10, 11]$ **Length:** 3 **Time:** $\mathcal{O}(n \log n)$ **Space:** $\mathcal{O}(n)$

LIS: Optimized Algorithm with Reconstruction

LIS with Binary Search and Reconstruction

Input: Array A of size n ($n \geq 1$)
Output: A longest increasing subsequence of A
Instructions:

1. $T \leftarrow [\,]$; $P \leftarrow [-1] * n$; $\ell \leftarrow 0$
2. For i from 1 to n do
3. $\quad k \leftarrow$ first position in T where $A[T[k]] \geq A[i]$ (binary search)
4. $\quad$ If no such position exists, $k \leftarrow \ell + 1$
5. $\quad$ If $k > 1$, then $P[i] \leftarrow T[k-1]$
6. $\quad$ If $k > \ell$, append i to T and set $\ell \leftarrow \ell + 1$
7. $\quad$ Otherwise, $T[k] \leftarrow i$
8. $S \leftarrow [\,]$; $i \leftarrow T[\ell]$
9. While $i \neq -1$ do
10. $\quad$ Prepend $A[i]$ to S; $i \leftarrow P[i]$
11. Return S

Time Complexity

- n binary searches: $\mathcal{O}(n \log n)$.
- Reconstruction: $\mathcal{O}(n)$.

$$\boxed{\mathcal{O}(n \log n)}$$

Space Complexity

Arrays T, P, and reconstructed LIS S each require at most n elements.

$$\boxed{\mathcal{O}(n)}$$

LIS: Applications in the Real World

1. Medical: Monitoring Patient Recovery

Problem
During rehabilitation, a patient's mobility is measured regularly (e.g., walking distance).

Measurements may fluctuate due to fatigue, pain, or temporary setbacks.

How can LIS help?
Find the longest sequence of measurements showing **continuous improvement in value**, even if the measurements are not consecutive.

Why is it useful?

- Identify sustained improvement patterns.
- Explore recovery progress despite fluctuations.

2. Industry: Production Monitoring

Problem
A factory records the number of products manufactured during each production shift.

Production varies because of machine stoppages, maintenance, or supply delays.

How can LIS help?
Find the longest sequence of shifts with **increasing production output**, while preserving their chronological order.

Why is it useful?

- Identify periods of improving productivity.
- Explore improvements despite temporary disruptions.

What Does LIS Actually Tell Us?

LIS identifies the **largest number of chronologically ordered observations with strictly increasing values**.

It helps reveal increasing patterns, but does not by itself measure the overall trend or explain the causes of improvement.

Contents

Core Principles of Dynamic Programming

Longest Increasing Subsequence (LIS)

Shortest Path

Edit Distance (Levenshtein)

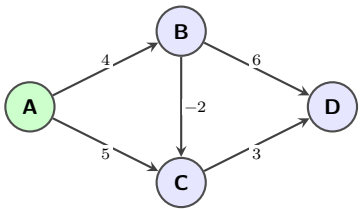
Shortest Path: Problem Statement

The Problem

Given a **weighted directed graph** $G = (V, E)$ and a source vertex s, find the **minimum total weight** of a path from s to every other vertex.

Edge weights may be **negative**, but we assume there are **no negative-weight cycles**.

Example: Source $s = A$



Shortest Paths from A

Vertex	Distance	Path
A	0	A
B	4	$A \to B$
C	2	$A \to B \to C$
D	5	$A \to B \to C \to D$

Why Is This Interesting?

The direct path $A \to C$ costs 5, but $A \to B \to C$ costs $4 + (-2) = 2$.
The shortest path is not necessarily the one with the fewest edges!

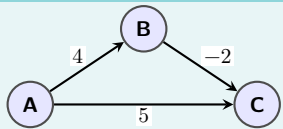
Shortest Path: What Is a Negative Cycle?

Definition

A **negative cycle** is a directed cycle whose **total edge weight is negative**.

Following this cycle repeatedly decreases the total cost of a path without limit.

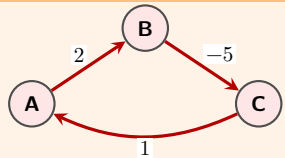
Negative Edge: Allowed



$A \to B \to C: \quad 4 + (-2) = 2$

Valid shortest path

Negative Cycle: Problematic



$2 + (-5) + 1 = \boxed{-2}$

Negative cycle!

Why Is a Negative Cycle a Problem?

Each traversal of the cycle decreases the path cost by 2: -2, -4, -6, -8, ... $\longrightarrow -\infty$
Therefore, **no finite shortest-path cost exists** for vertices reachable through this cycle.

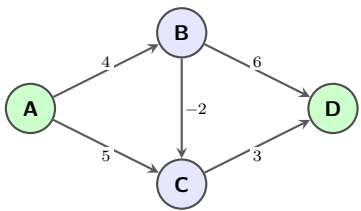
Shortest Path: Naive Solution

Brute Force: Explore All Possible Paths

For each destination vertex v, enumerate all **simple paths** from the source s to v, compute their total weights, and keep the minimum.

A **simple path** never visits the same vertex twice.

Example: Shortest Path from A to D



Enumerate All Paths

Path	Total weight
$A \to B \to D$	$4 + 6 = 10$
$A \to C \to D$	$5 + 3 = 8$
$A \to B \to C \to D$	$4 - 2 + 3 = 5$

Goal: Compare all candidate paths.

Why Is Brute Force Inefficient?

The number of simple paths can grow **factorially** with the number of vertices (e.g., $\Theta((|V| - 2)!)$ paths between two vertices in a complete directed graph).

Idea: Can we reuse previously computed shortest distances instead of exploring every possible path?

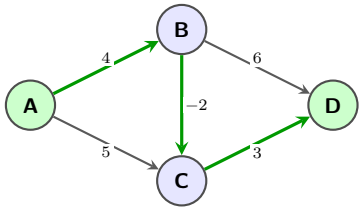
Shortest Path: Naive Solution

Brute Force: Explore All Possible Paths

For each destination vertex v, enumerate all **simple paths** from the source s to v, compute their total weights, and keep the minimum.

A **simple path** never visits the same vertex twice.

Example: Shortest Path from A to D



Enumerate All Paths

Path	Total weight
$A \to B \to D$	$4 + 6 = 10$
$A \to C \to D$	$5 + 3 = 8$
$\mathbf{A \to B \to C \to D}$	$\mathbf{4 - 2 + 3 = 5}$

Shortest path: $A \to B \to C \to D$

$$\boxed{d(A, D) = 5}$$

Why Is Brute Force Inefficient?

The number of simple paths can grow **factorially** with the number of vertices (e.g., $\Theta((|V| - 2)!)$ paths between two vertices in a complete directed graph).

Idea: Can we reuse previously computed shortest distances instead of exploring every possible path?

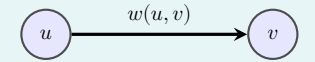
Shortest Path: Dynamic Programming (Bellman–Ford)

Key Idea: Reuse Previously Computed Distances

Instead of enumerating all possible paths, we progressively improve the shortest distances from the source s.

Let $D_k(v)$ be the shortest distance from s to v using **at most** k **edges**.

Edge Relaxation



If $w(u, v) = -2$:

$$dist[v] \leftarrow \min(10, 4 - 2)$$

$$\boxed{dist[v] = 2}$$

Dynamic Programming Recurrence

$$\boxed{D_k(v) = \min \left(\begin{array}{l} D_{k-1}(v), \\ \min\limits_{(u,v) \in E} (D_{k-1}(u) + w(u, v)) \end{array} \right)}$$

Two possibilities:

1. Keep the best distance using at most $k - 1$ edges.
2. Improve it by extending a path to u with the edge (u, v).

Initialization and Progression

Initially: $D_0(s) = 0$ and $D_0(v) = \infty$ for every $v \neq s$.

At each iteration k**:** Compute the best distances using at most k edges, based on iteration $k - 1$.

Bellman–Ford: From DP to an Iterative Algorithm

Algorithm: Repeated Edge Relaxation

1. **Initialize:** $dist[s] = 0$ and $dist[v] = \infty$ for all $v \neq s$.
2. **Repeat** $|V| - 1$ **times:** For every directed edge (u, v) with weight $w(u, v)$:

$$dist[v] \leftarrow \min(dist[v], dist[u] + w(u, v))$$

   Skip edges where $dist[u] = \infty$.
3. **Result:** $dist[v]$ contains the shortest distance from s to v.

Why $|V| - 1$ Iterations?

- A **simple path** visits each vertex at most once.
- With $|V|$ vertices, such a path contains at most $|V| - 1$ edges.
- Without reachable negative cycles, a shortest simple path always exists for every reachable vertex.

Detecting Negative Cycles

After the $|V| - 1$ rounds:

- Perform **one additional round** of edge relaxation.
- If any reachable distance can still decrease, a **negative cycle** is reachable from s.

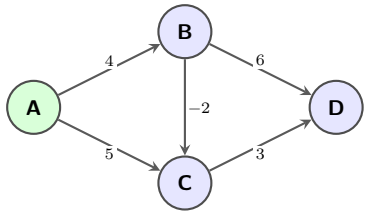
Complexity

Each round examines $|E|$ edges, and there are $|V| - 1$ rounds. Time: $\mathcal{O}(|V||E|)$ Space: $\mathcal{O}(|V|)$

Bellman–Ford: Step-by-Step Example

Example

Source: $s = A$. There are $|V| = 4$ vertices, so we perform $|V| - 1 = 3$ relaxation rounds.



Initialization: D_0

- Source A has distance 0.
- All other distances are ∞.
- No edges have been considered yet.

$$D_0 = [0, \infty, \infty, \infty]$$

Distance Table

Round	A	B	C	D
D_0	0	∞	∞	∞

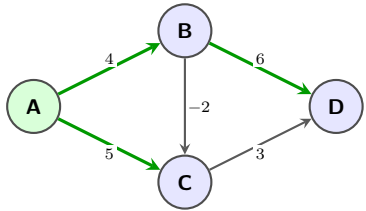
Observation

Each round allows paths with **one additional edge**. After $|V| - 1$ rounds, all shortest distances are known.

Bellman–Ford: Step-by-Step Example

Example

Source: $s = A$. There are $|V| = 4$ vertices, so we perform $|V| - 1 = 3$ relaxation rounds.



Distance Table

Round	A	B	C	D
D_0	0	∞	∞	∞
D_1	0	4	5	∞

Round 1: At Most 1 Edge

From A, we can reach:

$$D_1(B) = 0 + 4 = 4$$

$$D_1(C) = 0 + 5 = 5$$

$$D_1(D) = \infty$$

Result:

$$D_1 = [0, 4, 5, \infty]$$

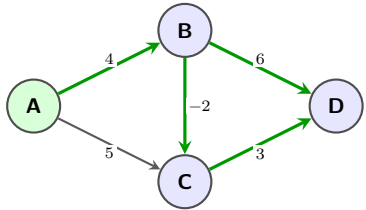
Observation

Each round allows paths with **one additional edge**. After $|V| - 1$ rounds, all shortest distances are known.

Bellman–Ford: Step-by-Step Example

Example

Source: $s = A$. There are $|V| = 4$ vertices, so we perform $|V| - 1 = 3$ relaxation rounds.



Distance Table

Round	A	B	C	D
D_0	0	∞	∞	∞
D_1	0	4	5	∞
D_2	0	4	2	8

Round 2: At Most 2 Edges

Using distances from D_1:

$$D_2(C) = \min(5, 4 - 2) = 2$$

$$D_2(D) = \min(4 + 6, 5 + 3) = 8$$

Result:

$$D_2 = [0, 4, 2, 8]$$

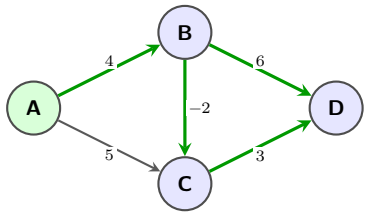
Observation

Each round allows paths with **one additional edge**. After $|V| - 1$ rounds, all shortest distances are known.

Bellman–Ford: Step-by-Step Example

Example

Source: $s = A$. There are $|V| = 4$ vertices, so we perform $|V| - 1 = 3$ relaxation rounds.



Distance Table

Round	A	B	C	D
D_0	0	∞	∞	∞
D_1	0	4	5	∞
D_2	0	4	2	8
D_3	0	4	2	5

Round 3: At Most 3 Edges

Using distances from D_2:

$$D_3(D) = \min(8, 2 + 3) = 5$$

Final distances:

$$\boxed{D_3 = [0, 4, 2, 5]}$$

Shortest path to D:

$$A \to B \to C \to D$$

Observation

Each round allows paths with **one additional edge**. After $|V| - 1$ rounds, all shortest distances are known.

Bellman–Ford: Dynamic Programming Algorithm

Bellman_Ford

Input: int V, int s, list $edges$ of (u, v, w)
Output: Array $dist$ or -1 if a reachable negative cycle exists

Instructions:

1. $prev \leftarrow [\infty] * V$
2. $prev[s] \leftarrow 0$
3. For k from 1 to $V - 1$ do
   $curr \leftarrow copy(prev)$
   For each (u, v, w) in $edges$ do
   If $prev[u] \neq \infty$ then
   $curr[v] \leftarrow \min(curr[v], prev[u] + w)$
   $prev \leftarrow curr$
4. For each (u, v, w) in $edges$ do
   If $prev[u] \neq \infty$ and $prev[u] + w < prev[v]$ then
   Return -1
5. Return $prev$

Bellman–Ford: Real-World Applications

1. Game AI and Decision-Making

Example: Finding an optimal sequence of actions in a game.

- Vertices represent game states.
- Edges represent possible actions.
- Each action may provide a reward or incur a penalty.

Action scores:

- **Positive (+):** Collecting coins, gaining points, or bonuses.
- **Negative (-):** Losing health, spending resources, or receiving penalties.

Goal: Find the sequence of actions maximizing the total score.

2. Transportation and Logistics

Example: Route planning with rewards.

- Locations are vertices.
- Edges represent possible routes.
- Each route may generate revenue or incur transportation costs.

Route scores:

- **Positive (+):** Delivery payments, bonuses, or subsidies.
- **Negative (-):** Fuel, tolls, or transportation fees.

Goal: Find the route maximizing the total profit.

Why Not Always Use Dijkstra?

- **Dijkstra:** Faster for graphs with nonnegative edge weights.
- **Bellman–Ford:** Handles negative weights and detects reachable negative cycles.

Contents

Core Principles of Dynamic Programming

Longest Increasing Subsequence (LIS)

Shortest Path

Edit Distance (Levenshtein)

Edit Distance (Levenshtein): Problem Statement

Problem

Given two strings s and t of lengths n and m, find the **minimum number of operations** required to transform s into t.

Allowed Operations

Each operation has a cost of **1**:

1. **Insertion:** Add a character.
2. **Deletion:** Remove a character.
3. **Substitution:** Replace one character with another.

Objective

Find the sequence of edits with the **smallest total cost**.

Example

$s =$ `"abcd"` $\quad t =$ `"bcfe"`

`abcd`	Initial string
$\downarrow$	Delete `a`
`bcd`	
$\downarrow$	Substitute `d` $\rightarrow$ `f`
`bcf`	
$\downarrow$	Insert `e`
`bcfe`	Final string

$$\boxed{\mathrm{ED}(s,t) = 3}$$

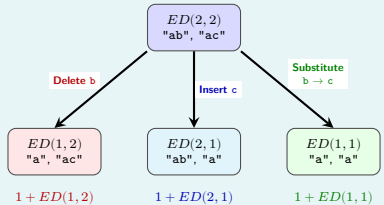
Edit Distance: Naive Recursive Approach

Brute-Force Idea

Compare the last characters of two strings. If they differ, try the three possible operations: **delete**, **insert**, or **substitute**, and recursively solve the smaller subproblems.

Example: $s =$ "ab", $t =$ "ac"

Goal: $ED(2,2)$ = distance between "ab" and "ac".



- Each node represents a **remaining subproblem**.
- Each branch adds **1 operation** before solving that subproblem.

Recursive Formula

Let $ED(i,j)$ be the edit distance between the first i characters of s and the first j characters of t.

Base cases:

$$ED(i,0) = i, \qquad ED(0,j) = j$$

If $s[i] = t[j]$**:**

$$ED(i,j) = ED(i-1, j-1)$$

Otherwise:

$$ED(i,j) = 1 + \min \begin{cases} ED(i-1,j) & \text{Delete} \\ ED(i,j-1) & \text{Insert} \\ ED(i-1,j-1) & \text{Substitute} \end{cases}$$

Operation	Remaining subproblem
Delete $s[i]$	$ED(i-1,j)$
Insert $t[j]$	$ED(i,j-1)$
Substitute	$ED(i-1,j-1)$

Why Is This Inefficient?

The recursion explores up to **three branches per call** and repeatedly solves the same subproblems. **Time:** $\mathcal{O}(3^{n+m})$ **Space:** $\mathcal{O}(n+m)$

Edit Distance: Dynamic Programming Solution

Key Idea: Bottom-Up Dynamic Programming

Instead of recursively solving the same subproblems, **store their results in a table** and compute larger subproblems from smaller ones.

DP Table

Define $dp[i][j]$ as the minimum number of edits needed to transform the **first** i **characters** of s into the **first** j **characters** of t.

Table dimensions:

$$(n+1) \times (m+1)$$

Base cases:

$$dp[i][0] = i \qquad dp[0][j] = j$$

- First column: delete all i characters.
- First row: insert all j characters.

Transition Rule

For $i = 1, \ldots, n$ and $j = 1, \ldots, m$:

If $s[i] = t[j]$**:**

$$dp[i][j] = dp[i-1][j-1]$$

Otherwise:

$$dp[i][j] = 1 + \min \begin{cases} dp[i-1][j] & \text{Delete} \\ dp[i][j-1] & \text{Insert} \\ dp[i-1][j-1] & \text{Substitute} \end{cases}$$

Computation order: Fill the table row by row, from top-left to bottom-right.

Complexity

Each of the nm cells is computed **once**, using at most three previously computed values. **Time:** $\mathcal{O}(nm)$ **Space:** $\mathcal{O}(nm)$

Edit Distance: Why Does Dynamic Programming Work?

Key Property: Optimal Substructure

An **optimal edit sequence** must use optimal solutions to its remaining subproblems. Otherwise, replacing a subsolution with a better one would improve the total cost.

Why Are Three Cases Enough?

Consider the last characters $s[i]$ and $t[j]$.

If they differ, an optimal alignment can end with:

1. **Delete** $s[i]$: $ED(i-1, j)$.
2. **Insert** $t[j]$: $ED(i, j-1)$.
3. **Substitute** $s[i]$: $ED(i-1, j-1)$.

If $s[i] = t[j]$, we can match them at **zero cost**.

Example: "ab" → "ac"

Since b ≠ c:

$$\text{Delete:} \quad 1 + ED(1,2) = 3$$
$$\text{Insert:} \quad 1 + ED(2,1) = 3$$
$$\text{Substitute:} \quad 1 + ED(1,1) = 1$$
$$\boxed{ED(2,2) = \min(3,3,1) = 1}$$

ab $\xrightarrow{\text{substitute b with c}}$ ac

Why Is the Result Optimal?

Every optimal edit alignment ends with one of these cases (or a character match). Since we consider **all possibilities** and select the **minimum**, we obtain the optimal solution.

Edit Distance: Dynamic Programming Algorithms

Top-Down: Recursion + Memoization

```
Input: Strings s, t (lengths n, m)
Output: Minimum edit distance

1. Initialize memo[0..n][0..m] = -1

2. Define ED(i, j):
   If i == 0: Return j
   If j == 0: Return i
   If memo[i][j] != -1:
      Return memo[i][j]
   If s[i] == t[j]:
      memo[i][j] = ED(i-1, j-1)
   Else:
      memo[i][j] = 1 + min(
         ED(i-1, j),
         ED(i, j-1),
         ED(i-1, j-1))
   Return memo[i][j]

3. Return ED(n, m)
```

Time: $\mathcal{O}(nm)$ **Space:** $\mathcal{O}(nm)$

Bottom-Up: Iteration + Tabulation

```
Input: Strings s, t (lengths n, m)
Output: Minimum edit distance

1. Create dp[0..n][0..m]

2. For i = 0 to n:
   dp[i][0] = i

3. For j = 0 to m:
   dp[0][j] = j

4. For i = 1 to n:
   For j = 1 to m:
      If s[i] == t[j]:
         dp[i][j] = dp[i-1][j-1]
      Else:
         dp[i][j] = 1 + min(
            dp[i-1][j],
            dp[i][j-1],
            dp[i-1][j-1])

5. Return dp[n][m]
```

Time: $\mathcal{O}(nm)$ **Space:** $\mathcal{O}(nm)$

Edit Distance: Tabulation and Backtracking

pikachu ⟶ pistachio

	ε	p	i	s	t	a	c	h	i	o
	0	1	2	3	4	5	6	7	8	9
p	1									
i	2									
k	3									
a	4									
c	5									
h	6									
u	7									

Table filling: row by row, then complete.

Step 1: Initialize

$dp[i][0] = i$ (delete all), and $dp[0][j] = j$ (insert all).

Delete $(+1)$ Insert $(+1)$

Substitute $(+1)$ Match $(+0)$

Edit Distance: Tabulation and Backtracking

pikachu ⟶ pistachio

	ε	p	i	s	t	a	c	h	i	o
	0	1	2	3	4	5	6	7	8	9
p	1	0	1	2	3	4	5	6	7	8
i	2									
k	3									
a	4									
c	5									
h	6									
u	7									

Table filling: row by row, then complete.

Step 2: Compute $dp[1][1]$

Compare p with p.

Three candidate costs:

$$\text{Delete} : 1 + 1 = 2$$
$$\text{Insert} : 1 + 1 = 2$$
$$\text{Substitute} : 1 + 0 = 1$$

Match: diagonal costs 0 instead of 1.

$$\boxed{dp[1][1] = 0}$$

Reuse values already computed.

Delete $(+1)$ Insert $(+1)$ Substitute $(+1)$ Match $(+0)$

Edit Distance: Tabulation and Backtracking

pikachu ⟶ pistachio

	ε	p	i	s	t	a	c	h	i	o
	0	1	2	3	4	5	6	7	8	9
p	1	0	1	2	3	4	5	6	7	8
i	2	1	0	1	2	3	4	5	6	7
k	3									
a	4									
c	5									
h	6									
u	7									

Table filling: row by row, then complete.

Step 3: Compute $dp[2][3]$

Compare i with s.

Three candidate costs:

$$\text{Delete} : 1 + 2 = 3$$
$$\text{Insert} : 1 + 0 = 1$$
$$\text{Substitute} : 1 + 1 = 2$$

$$\boxed{dp[2][3] = \min(3, 1, 2) = 1}$$

Reuse values already computed.

Delete (+1) Insert (+1)

Substitute (+1) Match (+0)

Edit Distance: Tabulation and Backtracking

pikachu ⟶ pistachio

	ε	p	i	s	t	a	c	h	i	o
	0	1	2	3	4	5	6	7	8	9
p	1	0	1	2	3	4	5	6	7	8
i	2	1	0	1	2	3	4	5	6	7
k	3	2	1	1	2	3	4	5	6	7
a	4	3	2	2	2	2	3	4	5	6
c	5									
h	6									
u	7									

Table filling: row by row, then complete.

Step 4: Compute $dp[3][4]$

Compare k with t.

Three candidate costs:

$$\text{Delete} : 1 + 2 = 3$$
$$\text{Insert} : 1 + 1 = 2$$
$$\text{Substitute} : 1 + 1 = 2$$

$$\boxed{dp[3][4] = \min(3, 2, 2) = 2}$$

Reuse values already computed. Delete (+1) Insert (+1) Substitute (+1) Match (+0)

Edit Distance: Tabulation and Backtracking

pikachu ⟶ pistachio

	ε	p	i	s	t	a	c	h	i	o
	0	1	2	3	4	5	6	7	8	9
p	1	0	1	2	3	4	5	6	7	8
i	2	1	0	1	2	3	4	5	6	7
k	3	2	1	1	2	3	4	5	6	7
a	4	3	2	2	2	2	3	4	5	6
c	5	4	3	3	3	3	2	3	4	5
h	6	5	4	4	4	4	3	2	3	4
u	7									

Table filling: row by row, then complete.

Step 5: Compute $dp[6][8]$

Compare h with i.

Three candidate costs:

$$\text{Delete} : 1 + 4 = 5$$
$$\text{Insert} : 1 + 2 = 3$$
$$\text{Substitute} : 1 + 3 = 4$$

$$\boxed{dp[6][8] = \min(5, 3, 4) = 3}$$

Reuse values already computed. Delete (+1) Insert (+1) Substitute (+1) Match (+0)

Edit Distance: Tabulation and Backtracking

pikachu ⟶ pistachio

	ε	p	i	s	t	a	c	h	i	o
	0	1	2	3	4	5	6	7	8	9
p	1	0	1	2	3	4	5	6	7	8
i	2	1	0	1	2	3	4	5	6	7
k	3	2	1	1	2	3	4	5	6	7
a	4	3	2	2	2	2	3	4	5	6
c	5	4	3	3	3	3	2	3	4	5
h	6	5	4	4	4	4	3	2	3	4
u	7	6	5	5	5	5	4	3	3	4

Table filling: row by row, then complete.

Step 6: Finish the table

All other cells are filled by exactly the same rule.

$$\boxed{dp[7][9] = 4}$$

Now trace an optimal path backward.

Delete $(+1)$ Insert $(+1)$ Substitute $(+1)$ Match $(+0)$

Edit Distance: Tabulation and Backtracking

pikachu ⟶ pistachio

		ε	p	i	s	t	a	c	h	i	o
		0	1	2	3	4	5	6	7	8	9
p		1	0	1	2	3	4	5	6	7	8
i		2	1	0	1	2	3	4	5	6	7
k		3	2	1	1	2	3	4	5	6	7
a		4	3	2	2	2	2	3	4	5	6
c		5	4	3	3	3	3	2	3	4	5
h		6	5	4	4	4	4	3	2	3	4
u		7	6	5	5	5	5	4	3	3	4

Current string:

p	i	k	a	c	h	o

Backtrack 1/5

Substitute u → o

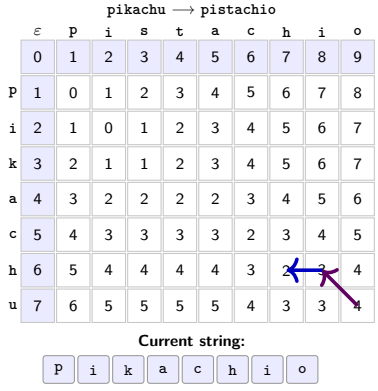
$dp[7][9] \rightarrow dp[6][8]$

pikachu → pikacho

Cost: 1 Delete (+1) Insert (+1)

Substitute (+1) Match (+0)

Edit Distance: Tabulation and Backtracking



Backtrack 2/5

Insert `i`

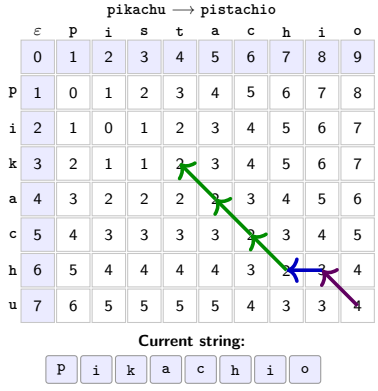
$dp[6][8] \rightarrow dp[6][7]$

`pikacho` → `pikachio`

Cost: 1 Delete $(+1)$ Insert $(+1)$

Substitute $(+1)$ Match $(+0)$

Edit Distance: Tabulation and Backtracking



Backtrack 3/5

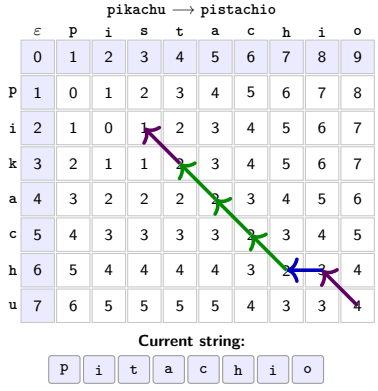
Match `h`, `c`, `a`

$dp[6][7] \rightarrow dp[3][4]$

`pikachio` (unchanged)

Cost: 0 Delete $(+1)$ Insert $(+1)$ Substitute $(+1)$ Match $(+0)$

Edit Distance: Tabulation and Backtracking



Backtrack 4/5

Substitute `k` → `t`

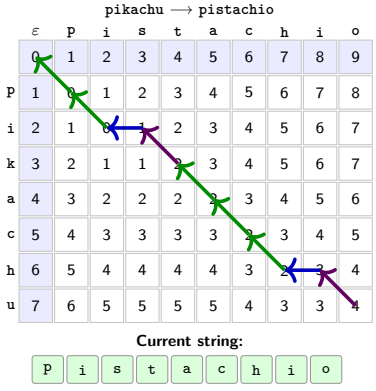
$dp[3][4] \rightarrow dp[2][3]$

`pikachio` → `pitachio`

Cost: 1 Delete $(+1)$ Insert $(+1)$

Substitute $(+1)$ Match $(+0)$

Edit Distance: Tabulation and Backtracking



Backtrack 5/5

Insert `s`; match `i`,`p`

$dp[2][3] \rightarrow dp[0][0]$

`pitachio` → `pistachio`

Cost: 1

Total: 4 edits. Delete (+1) Insert (+1) Substitute (+1) Match (+0)

Edit Distance: Where Is It Used?

Beyond Comparing Two Strings

Edit Distance measures how many operations are needed to transform one sequence into another. It is useful whenever we need to **compare, correct, or align sequences**.

1. Spell Checking & Autocorrect

Suggest words close to a misspelled input.

`recieve` $\longrightarrow$ `receive`

Used in text editors, search engines, and smartphone keyboards.

2. DNA Sequence Comparison

Compare biological sequences to identify mutations and similarities.

`ACGT` $\longrightarrow$ `AGGT`

One substitution may represent a mutation.

3. Search Engines & Fuzzy Matching

Find relevant results even when the query contains typing errors.

`algoritm` $\longrightarrow$ `algorithm`

Useful for approximate string matching and database record linkage.

4. Speech Recognition Evaluation

Compare a predicted transcription with the correct sentence.

`I like cats` $\longrightarrow$ `I like cat`

Word-level edit distance is used to compute the **Word Error Rate (WER)**.

Session Summary: Dynamic Programming

What Is Dynamic Programming?

Dynamic Programming (DP) solves a problem by breaking it into **smaller overlapping subproblems**, solving each one **only once**, and reusing its result.

When Can We Use DP?

Two key properties:

1. **Optimal substructure:** An optimal solution can be built from optimal solutions to smaller subproblems.
2. **Overlapping subproblems:** The same subproblems appear repeatedly.

General strategy:

1. Define the DP state.
2. Derive the recurrence.
3. Identify the base cases.
4. Compute and store the results.

Two Implementation Approaches

Top-Down (Memoization)

- Start from the original problem.
- Solve recursively and cache results.

Bottom-Up (Tabulation)

- Start from the smallest subproblems.
- Fill a table in dependency order.

Both approaches use the same recurrence and avoid redundant computations.

Key takeaway:

Find the recurrence, store intermediate results, and build the final solution from smaller ones.